Mirror Encryption: Security Architecture

Request for comments

AJS Consult LLC. Editor: Aaron Soellinger.

Contents

1 . . . 5

2 Executive summary . . . 7

3 1. Document control . . . 9

4 2. Scope . . . 11

5 3. System overview . . . 13

6 4. Threat model . . . 15

7 5. Security goals and non-goals . . . 17

8 6. Cryptographic inventory . . . 19

9 7. Key hierarchy . . . 21

10 8. Key management lifecycle . . . 23

11 9. Protocol flows . . . 27

11.1 9.1 Enrollment: the client's first run . . . 27

11.2 9.2 Opening the web board . . . 27

11.3 9.3 Connecting an AI assistant (OAuth 2.1) . . . 28

11.4 9.4 An AI asks a question . . . 29

11.5 9.5 Arrivals while nobody is calling . . . 30

11.6 9.6 Debug access . . . 31

11.7 9.7 A lost device, and a new one . . . 32

11.8 9.8 Disconnect, sign out, delete . . . 33

12 10. Searching encrypted data . . . 35

13 11. Performance . . . 37

14 12. Security analysis . . . 39

15 13. Residual risks and known limitations . . . 41

16 14. Verification and transparency . . . 43

17 15. Implementation status and roadmap . . . 45

18 16. Questions for reviewers . . . 49

19 17. Vulnerability disclosure 51

20 18. References and glossary 53

Appendices 55

A Appendix A. Submission process 57

1.

Nov 11, 2026 · @aaron soellinger

A request for comments (RFC) on encrypting private data mirrors so that only the user-approved devices and AIs can read them.

2. Executive summary

Today, AI assistants connect directly to a user's private correspondence via a variety of partially and fully supported, secure (and insecure) connectors. Under this status quo, AI providers are empowered to act as fiduciary over this data, to the end of enabling bleeding edge automation and research capabilities to their users. Simply put, users should not need to sign over their data ownership rights in exchange for access to the best AIs. This document proposes a security architecture for a messaging mirror that can be securely, safely and privately operated by the user themselves or by a trusted 3rd party.

The problem. Data ownership is one of the biggest impediments to the proliferation of beneficial AI, especially in security, safety and privacy critical industries. Many individuals and organizations are unwilling to cede their data rights to AI companies. There currently exists no end-to-end security reference architecture to enable individuals or organizations to securely, safely and privately host their own data mirrors in a way that is integrated and compatible with AI assistants.

The alternative. Without a community effort to develop a security, privacy and safety standard around private data, the most likely outcome is for AIs to continue centralizing power using this strategy. Private 3rd party companies hosting closed source messaging mirrors require the user to trust that bond. The technical requirements of building such data management systems require seasoned, credentialed software engineering veterans. Vibe-coded systems from novices don't cut it.

Why this document. I am an engineer with more than 15 years in industry and independent research in machine learning. For the last six months I have worked on delivering AI to businesses safely, securely and privately, and I built a system that does it over a customer's own messages. Every customer contract turns on the same question: how is their data protected? The industry would benefit from answering that question once, together. This document is my steel-man architecture, the strongest version I can make, for others to attack and improve.

What is asked of you. Attack the design. The threat model (section 4), the search leakage (section 10) and the fourteen open questions (section 16) matter most.

3. 1. Document control

This is version 0.1, a request for comments. It is owned by its author personally, not by the company that runs the service.

Field	Value
Status	Request for comments. A design proposal: nothing in sections 6 to 10 is built.
Version	0.1, 11 November 2026
Owner and editor	Owned by AJS Consult LLC, © 2026 AJS Consult LLC. Edited by Aaron Soellinger.
Interests	The editor founded and owns AJS Consult LLC, which owns the brands PRCVD.AI, a machine learning consultancy, and MatterBoard, a safe, secure and private messaging mirror optimized for AI tool use. The system this document describes is MatterBoard's, so the company and the editor have a financial interest in it. AJS Consult LLC is actively engaged in commercial programs to safely, securely and privately manage data in AI pipelines.
How it was written	Drafted with help from Claude, an AI model, from a read of the product's source code on 11 November 2026. The author is responsible for every claim in it.
License	CC BY 4.0. You may share, quote and adapt it, with credit to AJS Consult LLC.
Commitment	Nothing here commits AJS Consult LLC until it ships and the company's privacy policy says so.
Version of record	The copy at this address. Each revision is dated and listed below, and earlier versions stay readable in the document's history.

Comments. Anyone may comment, on any part of this document. Your comments stay yours. By posting one, you let the author use the idea without obligation or payment, and you are credited by name if you ask. Please leave out confidential information and anything about other people.

Security problems in the running service go privately to the contact in section 17, never in a comment here.

Open question: the date the comment period closes.

Version	Date	Change
0.1	11 November 2026	Initial commit.

4. 2. Scope

This document covers the server-side copy of each account's data, which the service calls the mirror, and every credential that can open it.

The system. The client, a desktop app, reads the user's mail, messages, calendars, contacts and files on the device and uploads a copy to the server: the mirror. AI assistants the user connects, today Claude and ChatGPT, query the mirror over MCP. A web board shows it in a browser. The server also fetches Slack itself, receives inbound email, and runs a Telegram bot.

In scope	What it covers
Content at rest	Messages, chats, contacts, calendar events, attachments, contact photos and uploads, plus records derived from them (triage, matters, people, traces)
Keys and tokens	The account key, its locked copies, every token a holder presents, and the source tokens for Google and Slack
Access paths	The client, AI assistants over MCP and OAuth 2.1, the web board, and company support
Search	Matching and regex search over encrypted content
Backups	Cloud SQL point-in-time recovery and daily backups

Built today versus proposed. Today the mirror's content is stored in plaintext columns in Cloud SQL (Postgres 15) and objects in Cloud Storage, under Google's default encryption at rest. All traffic uses TLS. Tokens are stored as SHA-256 hashes. Google grants are encrypted with one server-wide key. Today, the service's administrators can read or change any account's mirror through the service's own routes, and the service keeps no audit record of which administrator did so. Everything in sections 6 to 10 is proposed and not built.

Phasing. Accounts that use only the client switch first. The server does no background AI work for them (triage, filing, briefs), so it reads their content only while answering a request or taking an upload. Accounts that use the web board's background features follow once section 16′s questions on them are settled.

Out of scope: the security of the user's device beyond the client's use of the OS keystore; what AI providers do with answers they receive; the original sources (Gmail, Microsoft 365, iMessage, WhatsApp, Slack, Telegram); transport security beyond requiring TLS; and availability.

5. 3. System overview

The server holds K only inside a request that presents a holder's token and secret. Storage, backups and logs hold ciphertext, keyed tags and token hashes.

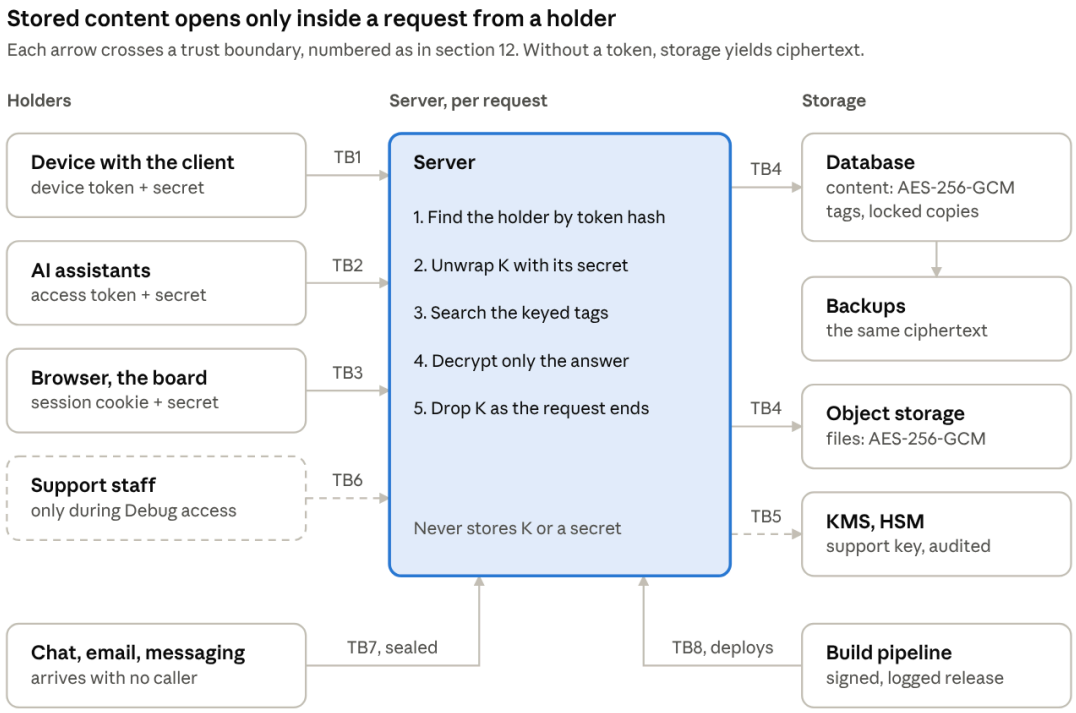


Figure 5.1: system overview · 4 holders, 4 stores, 8 trust boundaries

The server is a FastAPI application on Python 3.11, on Google Cloud, behind TLS. Dashed parts exist only while the user has Debug access on.

6. 4. Threat model

The design defends stored data against anyone who does not hold a token, the operator included. It does not defend data in server memory during an authorized request until verification level 3 (section 14).

Assets

Asset	Where it lives
Message content: text, subjects, attachments	Postgres, Cloud Storage, backups
Contacts and the identity graph: handles, addresses, names, aliases	Postgres
Calendar events	Postgres
Source tokens: Google refresh token, Slack tokens	Postgres
The account key and its locked copies	The device's OS keystore; locked copies in Postgres
Holder tokens: device, access, refresh, session, personal access	Each holder; only hashes on the server
Metadata: timestamps, sizes, counts, which rows share a search fragment	Postgres

Adversaries

#	Adversary	Capabilities	Outcome under this design
A1	Storage thief	Reads Cloud SQL, a database dump, a backup or a storage bucket	Ciphertext, keyed tags and token hashes. Learns metadata and tag overlap (section 10).
A2	Curious or compelled operator	Operator staff with database and admin access who do not change the code	Ciphertext, unless the user has turned on Debug access. Every support use is logged by Cloud KMS and shown to the user.
A3	Compromised server or malicious code change	Runs code on the server while requests are served	Sees the key and plaintext of every account that makes a request while it is present. Not defended until level 3.
A4	Token thief	Steals one holder's token from an AI provider, a browser or a device	That holder's access, until the token is revoked or expires.
A5	Another user of the service	Authenticated to a different account	Ciphertext that does not decrypt under their key, even if a bug

#	Adversary	Capabilities	Outcome under this design
			serves them another account's rows.
A6	AI provider	Receives what it asks for	Sees those answers. A non-goal (section 5).
A7	Lawful demand served on the operator	Compels data the company holds	Ciphertext, unless a holder is calling or Debug access is on. A compelled code change is A3.
A8	Malware on the user's device	Runs as the user	Has the key. Out of scope.

Trust assumptions

- The device is not compromised when the client first runs, and the OS keystore protects the key at rest.
- The Cloud KMS HSM never releases the support key's private half.
- Google Cloud's infrastructure does not attack the running server. Level 3 removes this assumption for the server.
- AES-256-GCM, HKDF-SHA256, HMAC-SHA256, X25519 and RSA-OAEP are secure at the parameters in section 6.
- Every connection uses TLS 1.2 or later.
- The account holder, signed in, is the person who approves a new AI on the consent page.

7. 5. Security goals and non-goals

This is server-side encryption under keys the user's holders control. It is not end-to-end encryption: the server decrypts, in memory, for each authorized request.

Goals

#	Goal	Where it is met
G1	Without a holder's token, stored content, contacts and calendars are ciphertext to everyone, operator staff and anyone with the database, backups or storage included.	Sections 6, 7, 10
G2	Each holder has its own locked copy. Revoking a holder deletes one row and ends its access at once.	Sections 7, 8
G3	Support can read an account only after the user turns on Debug access. It is time-limited, revocable, logged by a party other than the operator, and every use is shown to the user.	Section 9.6
G4	The operator cannot re-read the user's original sources: the Google and Slack tokens are locked under the account key.	Section 8
G5	Deleting the account deletes every live copy of the key, and the Vault deletes K from the Keychain. Backups keep locked copies until they expire, within 30 days.	Section 9.8
G6	One account's ciphertext and tags cannot be read or matched with another account's keys.	Sections 6, 10
G7	An authorized AI's answer costs about what it costs today.	Section 11
G8	People outside the company can check the code, the build and, at level 3, the running server.	Section 14

Non-goals

#	Not promised	Why
N1	Protecting plaintext in server memory during an authorized request from a compromised server	Needs attestation, level 3

#	Not promised	Why
N2	Controlling what an AI provider does with an answer	Outside the operator's control
N3	Hiding metadata: timestamps, sizes, counts, access times, and which rows share a keyed fragment	Needed for search and ordering (section 10)
N4	Protecting backups taken before cutover	They expire within 30 days
N5	Defending against malware on the user's device	The device holds the key
N6	Keeping the mirror readable if every holder is lost	No recovery key exists. The mirror refills from the original sources.

8. 6. Cryptographic inventory

Every primitive is a published standard from a maintained library: the platform's native cryptography library on the device, and pyca/cryptography (Python 3.11) on the server. Nothing is a custom construction except the search tags in section 10.

Notation used from here on. K is an account's key. For a holder h, T_h is its token, s_h its unlock secret, and C_h its locked copy of K. A holder presents T_h and s_h together. The server stores SHA-256(T_h) and C_h, never T_h, s_h or K.

Purpose	Primitive	Parameters	Library	Standard	Status
Account key K	Random generation	256 bits	Platform cryptography library	—	Proposed
Content in Postgres	AES-256-GCM	96-bit random nonce, 128-bit tag; associated data binds account, table, column and row	pyca/cryptography	NIST SP 800-38D	Proposed
Files in Cloud Storage	AES-256-GCM, chunked stream	Per-chunk nonce and position; chunk size open	pyca/cryptography	NIST SP 800-38D	Proposed
Unlock secret s_h	Random generation	256 bits	Platform cryptography library; Python `secrets`	—	Proposed
Key-encryption key for C_h	HKDF-SHA256	IKM s_h; info names the holder and the version	pyca/cryptography	RFC 5869	Proposed
Locked copy C_h	AES-256 key wrap with padding	256-bit key-encryption key	pyca/cryptography	RFC 5649	Proposed
Holder lookup	SHA-256 of T_h	Unsalted: T_h is 240 random bits	Python `hashlib`	FIPS 180-4	Built
Search and identity tag keys	HKDF-SHA256	IKM K; one info label per purpose	pyca/cryptography	RFC 5869	Proposed
Search and identity tags	HMAC-SHA256, truncated	64 bits	Python `hmac`	RFC 2104	Proposed
Sealed arrivals	HPKE, base mode	DHKEM(X25519, HKDF-	Open (question 9)	RFC 9180, RFC 7748	Proposed

Purpose	Primitive	Parameters	Library	Standard	Status
		SHA256), HKDF-SHA256, AES-256-GCM			
Support copy of K	RSA-OAEP	4096-bit key, SHA-256, in a Cloud KMS HSM	Cloud KMS	RFC 8017	Proposed
AI authorization	OAuth 2.1 with PKCE	S256 challenge; authorization code 300 s, single use	Server's own	RFC 7636, RFC 9700	Built
Google grants, today	Fernet (AES-128-CBC with HMAC-SHA256)	One key for all accounts	pyca/cryptography	Fernet spec	Built; replaced by K for source tokens
Release signing	Sigstore cosign, Rekor log	—	Sigstore	SLSA	Proposed
Server attestation	AMD SEV-SNP confidential VM	Image digest in the attestation	Google Confidential Space	—	Proposed

9. 7. Key hierarchy

One account key, K, encrypts everything, and each holder reaches it only through a copy wrapped under its own secret.

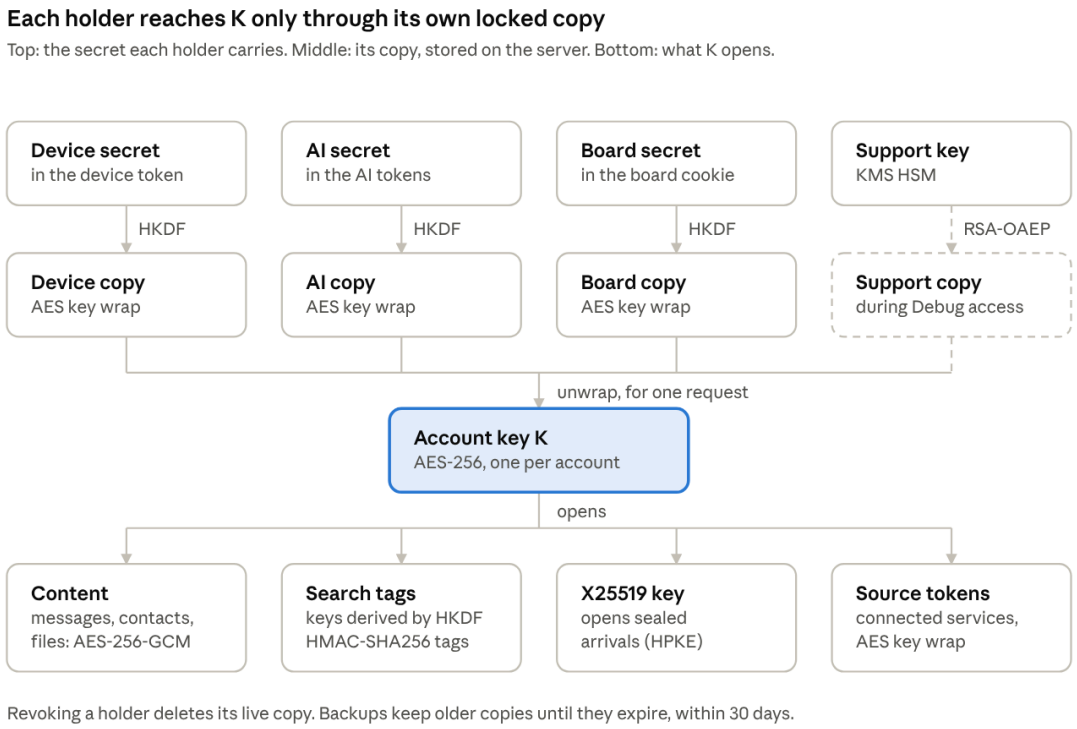


Figure 9.1: key hierarchy · 4 holders, 1 account key, 4 things it opens

Deleting a copy revokes one holder. A personal access token, when built, is one more holder in the top row.

10. 8. Key management lifecycle

Every credential that can reach K follows the phases of NIST SP 800-57 Part 1, below. Revoking any holder is deleting one row. Rows describe the proposal; the token formats marked built exist today.

Credential	Generated	Stored	Distributed	Used	Rotated	Revoked and destroyed
Account key K	The client's first run, with the platform's cryptography library	OS keystore, the source of truth, bound to this device and readable only while it is unlocked. The server holds only locked copies.	Never in the clear; only as locked copies	Server memory, for one request; derives the tag keys	Open (question 3)	Account deletion deletes every live copy, and the Vault deletes K from the Keychain; backups age out within 30 days. Losing every holder makes the mirror unreadable.
Locked copy C_h and unlock secret s_h	When a holder is admitted: sign-in, AI consent, a board session, Debug access, a personal access token	C_h in Postgres with its holder, creation time and expiry. s_h only inside the holder's token.	s_h travels with every request	HKDF(s_h) unwraps C_h for that request	With the holder's token: a new secret rewraps the copy	Deleting C_h ends that holder's access at once. A backup taken earlier still holds it until the backup expires.
X25519 keypair	With K, on the device	Public half in Postgres, in the clear. Private half encrypted under K.	Public half to every producer that seals arrivals	Seal: Slack sync, inbound email, Telegram. Open: the next holder's call.	With K, after opening every sealed arrival	Account deletion
Search and identity tag keys	Derived from K by HKDF on	Never stored	Never	Computing tags for writes and queries	With K; every tag is recomputed	With K

Credential	Generated	Stored	Distributed	Used	Rotated	Revoked and destroyed
	each request					
Support key	Once, in a Cloud KMS HSM, by infrastructure code	The private half never leaves the HSM	The public half to the server	KMS decrypt of a support copy, by staff with the IAM role and two-step sign-in; logged in Cloud Audit Logs	KMS key versions; support copies rewrap on next use	KMS key-version destruction
Support copy	When the user turns on Debug access	Postgres, with its expiry	Never leaves; the server sends it to KMS and gets K back for one request	Each support request	—	At expiry (72 hours suggested), or when the user turns it off
Device token	At sign-in, with Google or an emailed code. Built: 240 random bits.	OS keystore; the server keeps SHA-256	`Authorization: Bearer` over TLS	Every client request	Open (question 13)	Sign out, removal from Control, account deletion
Access token (AI)	`/token`, after consent. Built.	The AI provider; the server keeps SHA-256	Bearer header on `/mcp` only	Each MCP call	Each refresh	Disconnect from Control, account deletion
Refresh token (AI)	`/token`. Built: single use.	The AI provider; the server keeps SHA-256	`/token` only	Renewing the access token	Spent and replaced on every use	Deleted with the AI's access token and copy on disconnect
Session cookie (board)	Redeeming a connect code, or signing in on the web. Built.	An HttpOnly, Secure, SameSite=Lax cookie only	The cookie	Each board request	A new session replaces the previous one for the same purpose	Sign out, revoking its parent device, account deletion
Connect code	The client asks for	A server row until	The URL fragment, which	Once	—	Deleted on redemption; ex-

Credential	Generated	Stored	Distributed	Used	Rotated	Revoked and destroyed
	one. Built: 256 bits.	redeemed or expired	browsers do not send to servers			pires after 60 seconds
Personal access token	Minted from Control by the signed-in owner, for a script	The script; the server keeps SHA-256, its scopes and its expiry	Bearer header on a REST surface	Each request, like any holder	Mint a new one, then delete the old one	At its expiry, or deleted from Control
Source tokens (Google, Slack)	When the user connects the source	Wrapped under K	Never	Only inside a request that holds K	When the provider rotates them	Deleted, and revoked with the provider, on disconnect

11. 9. Protocol flows

The eight flows below are the whole proposal: in each, K is unwrapped only inside a request that presents a holder's secret. Notation is from section 6. Steps marked "built" run that way today.

11.1 9.1 Enrollment: the client's first run

1. Client ⟶ Server: sign in with Google or an emailed code. Built.
2. Server ⟶ client: device token T_m. The server stores SHA-256(T_m). Built.
3. Client: generates K, an X25519 keypair (pk, sk), and the device's unlock secret s_m.
4. Client: stores K, T_m and s_m in the OS keystore.
5. Client ⟶ Server: C_m = Wrap(HKDF(s_m), K), pk, and sk encrypted under K.
6. Server: stores them. K, sk and s_m never leave the device in the clear.

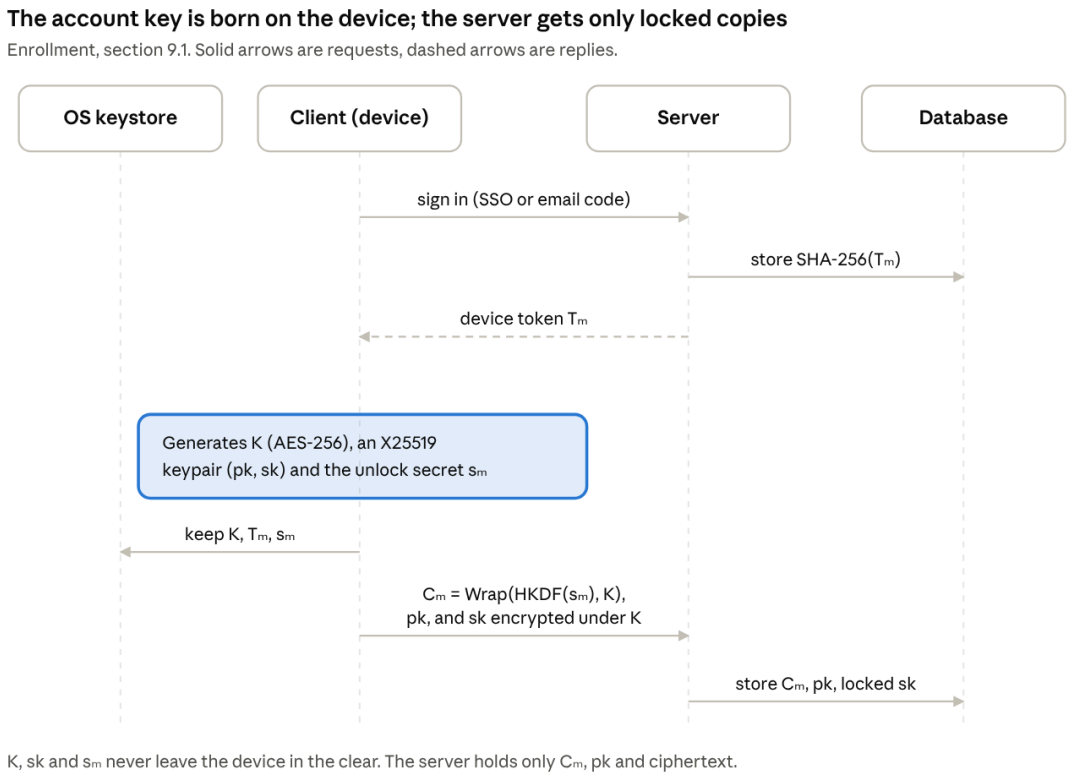


Figure 11.1: enrollment sequence · 4 parties, built vs proposed

The first three messages run today; everything below the line is proposed. K is created on the device, and only locked copies cross to the server.

11.2 9.2 Opening the web board

1. Client ⟶ Server: request a connect code, presenting T_m and s_m. Built, without s_m.
2. Server ⟶ client: a 60-second, single-use connect code. Built.
3. Client: opens the browser with the code in the URL fragment. Built.
4. Browser ⟶ Server: redeem the code, from a trusted Origin. Built.

5. Server: makes the session's secret s_e and its copy C_e.
6. Server ⟶ Browser: an HttpOnly session cookie carrying T_e and s_e.

Whether C_e is made at step 2 or step 5 is open (question 6). Made at step 5, a code that expires unused leaves no copy behind, but the code must then carry a secret that opens K.

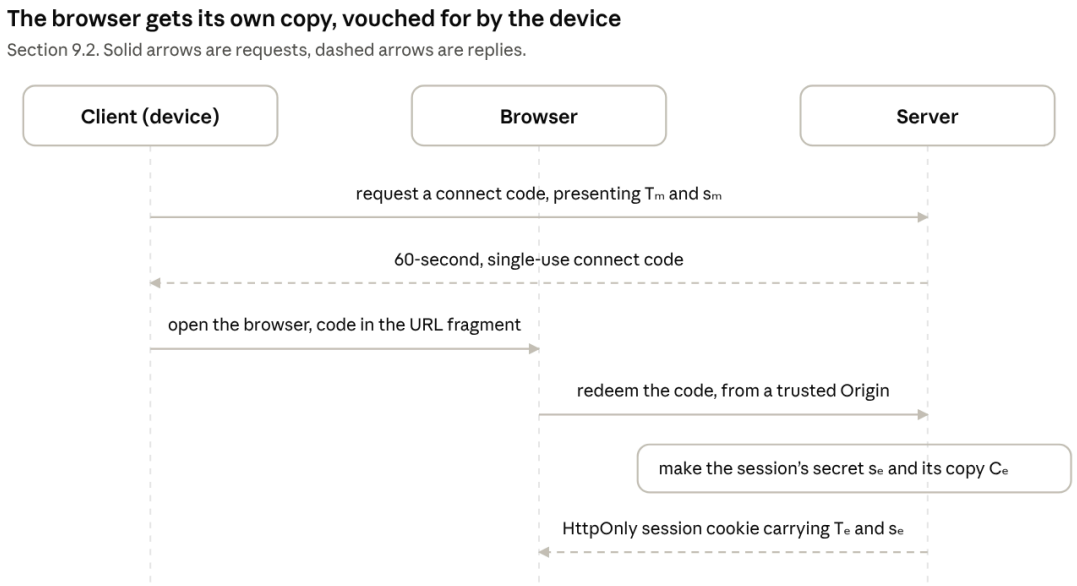


Figure 11.2: opening the board · 3 parties, 6 steps

The device vouches for the browser with a one-use code; the browser then holds its own secret in an HttpOnly cookie.

11.3 9.3 Connecting an AI assistant (OAuth 2.1)

1. AI ⟶ Server: dynamic client registration, then a redirect to `/authorize` with a PKCE S256 challenge. Built.
2. User, signed in on the board, approves on the consent page. Built.
3. Server: unwraps K through the session's copy, makes s_a for the AI, stores C_a.
4. Server ⟶ AI: an authorization code, 300 seconds, single use. Built.
5. AI ⟶ Server: `/token` with the code and the PKCE verifier. Built.
6. Server ⟶ AI: an access token and a refresh token, each carrying s_a.
7. On renewal, AI ⟶ Server: `/token` with the refresh token, which carries s_a, so C_a stays openable (question 7).

Today the redirect allowlist admits Claude, ChatGPT and loopback clients only.

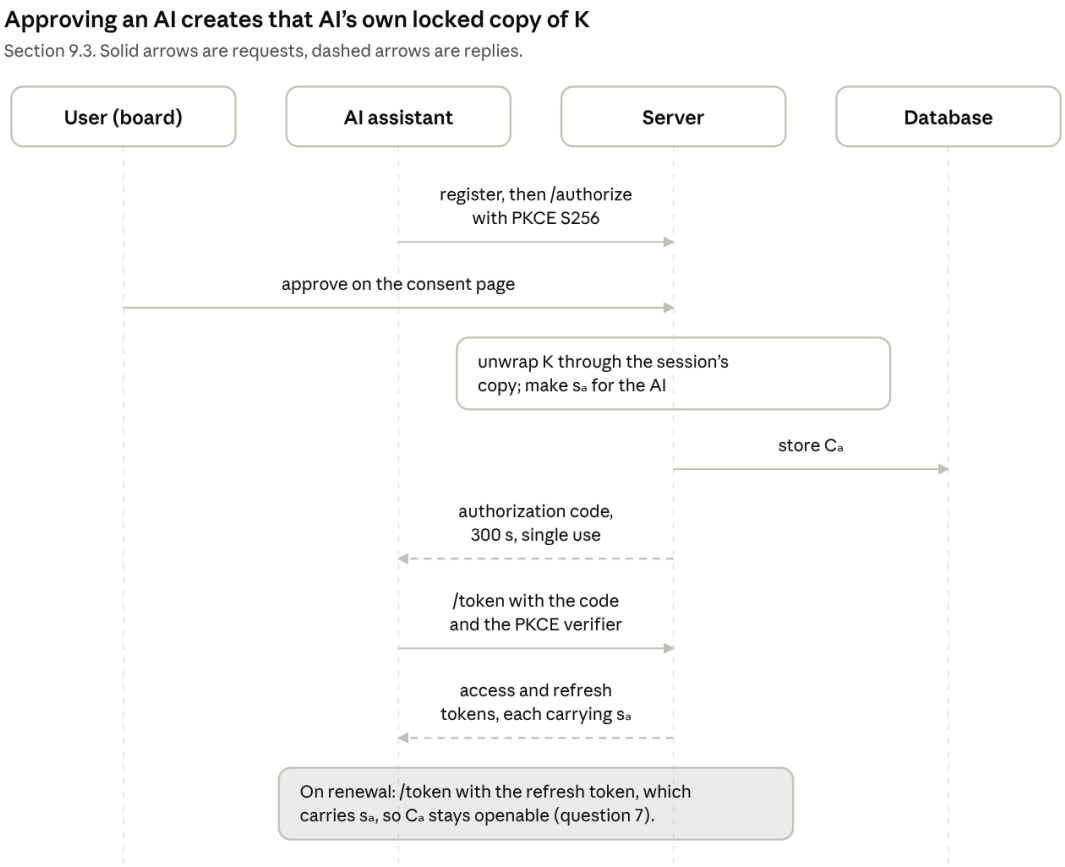


Figure 11.3: connecting an AI · 4 parties, OAuth 2.1 with PKCE

The copy is made inside the user's signed-in session, so approving the AI is the only step the user takes.

11.4 9.4 An AI asks a question

1. AI ⟶ Server: an MCP call, `Authorization: Bearer` carrying T_a and s_a. Built, without s_a.
2. Server: finds the holder by SHA-256(T_a), and its copy C_a. Built, without C_a.
3. Server: K = Unwrap(HKDF(s_a), C_a), held in a request-scoped context. A route that needs K and lacks it fails; it never falls back to plaintext.
4. Server ⟶ Postgres: look up the query's keyed tags in the GIN index (section 10).
5. Postgres ⟶ Server: candidate rows, still ciphertext.
6. Server: decrypts only the candidates, runs the real regex, and returns the answer.
7. The request ends. K and the plaintext are dropped; nothing is written or logged in the clear.

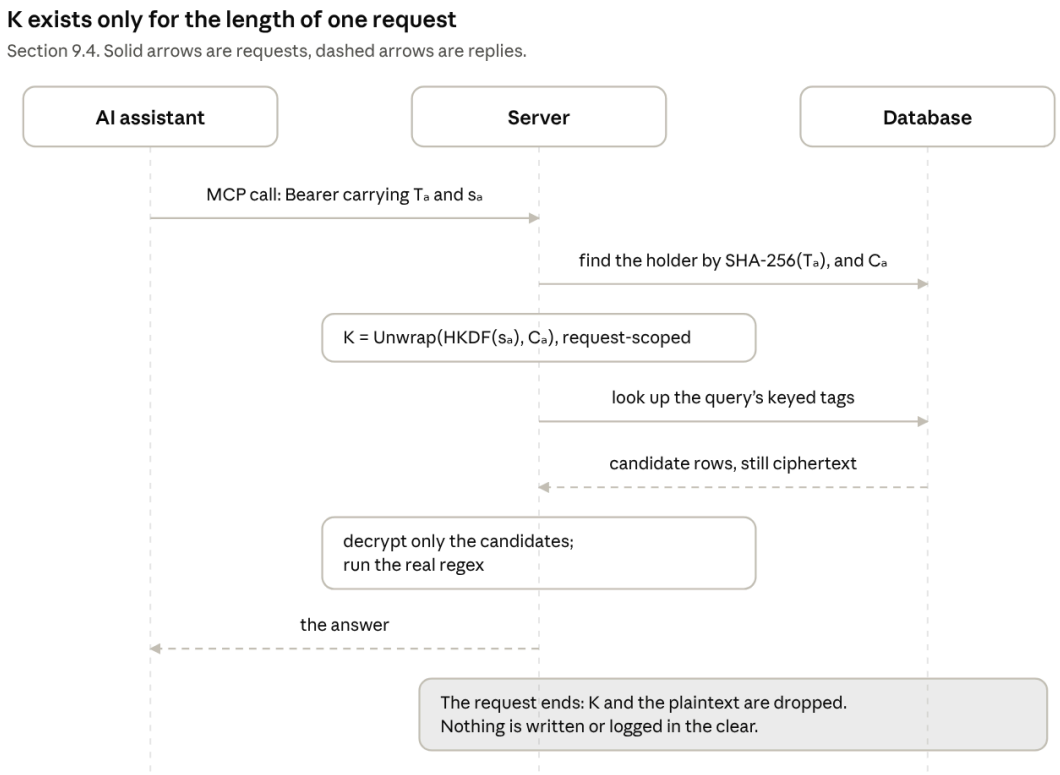


Figure 11.4: an AI asks · 3 parties, one request

Only the candidate rows are decrypted, and K is dropped when the request ends.

11.5 9.5 Arrivals while nobody is calling

1. Slack sync, the inbound email webhook or the Telegram bot ⟶ Server: a new message. Built.
2. Server: seals it to pk with HPKE and stores the sealed record. The plaintext exists in memory only while it is sealed.
3. On the next holder's call, the server unwraps K, decrypts sk, opens the sealed records, tags them and re-encrypts them under K.

The arrivals queue, which runs on Google Pub/Sub in development, carries message ids only, never content.

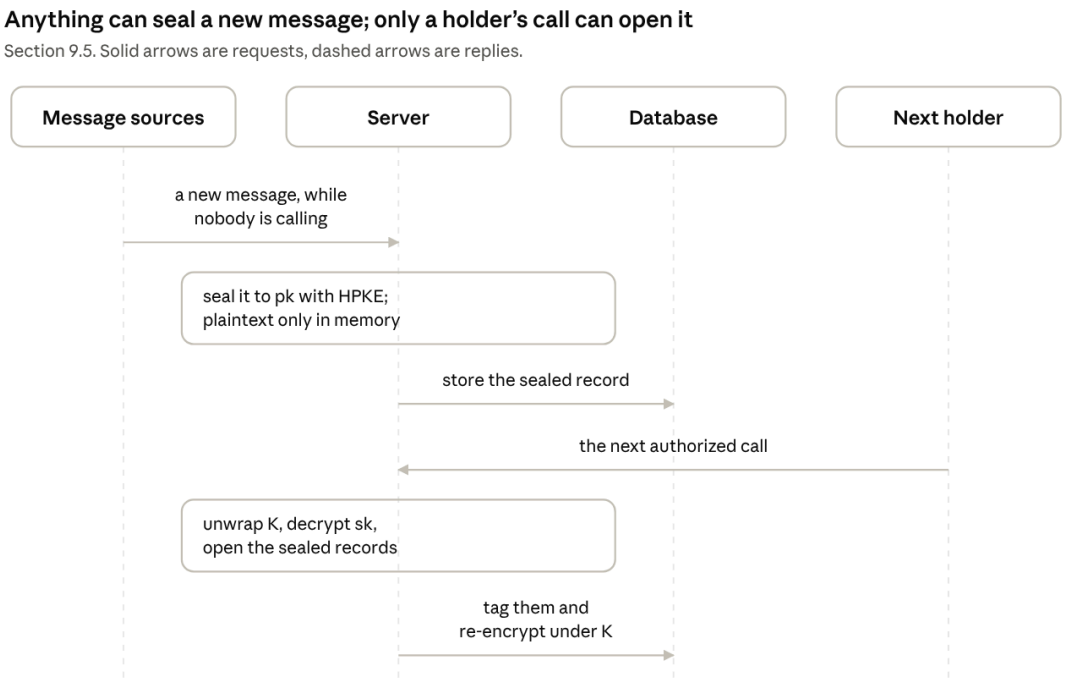


Figure 11.5: arrivals · 4 parties, seal now, open later

New mail keeps arriving without the server ever holding K; it waits sealed until a holder calls.

11.6 9.6 Debug access

1. User, signed in (the session holds K) ⟶ Server: turn on Debug access, with an end time and an optional reason.
2. Server: C_s = RSA-OAEP(support public key, K), stored with its expiry.
3. Support staff, with the IAM role and two-step sign-in ⟶ Server: a support request.
4. Server ⟶ Cloud KMS: decrypt C_s. KMS ⟶ Server: K, for that request. Cloud Audit Logs records the call.
5. Server ⟶ User: the use appears in the user's list, read from the audit log.
6. At the end time, or when the user turns it off, the server deletes C_s.

Cloud KMS does not know the end time; the server enforces it by deleting the copy (question 8).

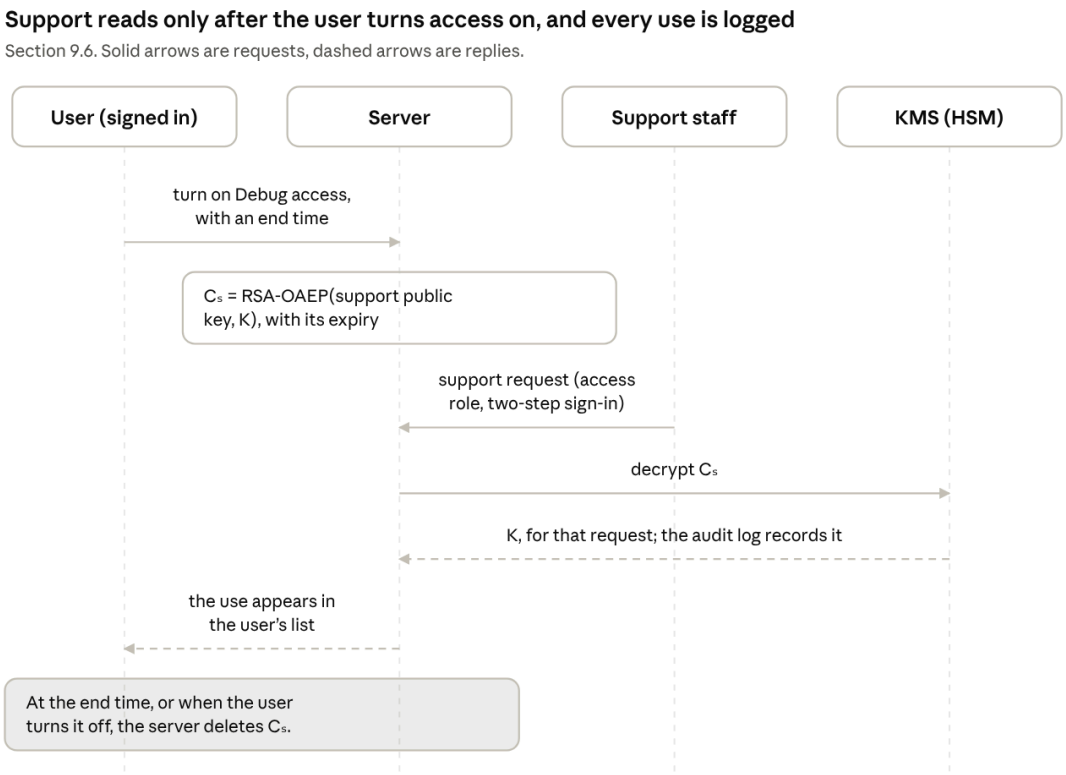


Figure 11.6: Debug access · 4 parties, user-granted and logged

Support has no copy until the user turns access on, and the KMS audit log records each use.

11.7 9.7 A lost device, and a new one

1. User, on the board → Server: remove the lost device. The server deletes C_m and revokes T_m.
2. New device → Server: sign in and ask to join. The new device shows a short code.
3. User, on an existing holder: compares the code and approves.
4. Server: makes a copy for the new device under its own new secret.
5. If no holder is left, the mirror cannot be read. The new device makes a new K and refills the mirror from the original sources.

How a second device joins, and whether it also keeps K locally, is not specified yet (question 4).

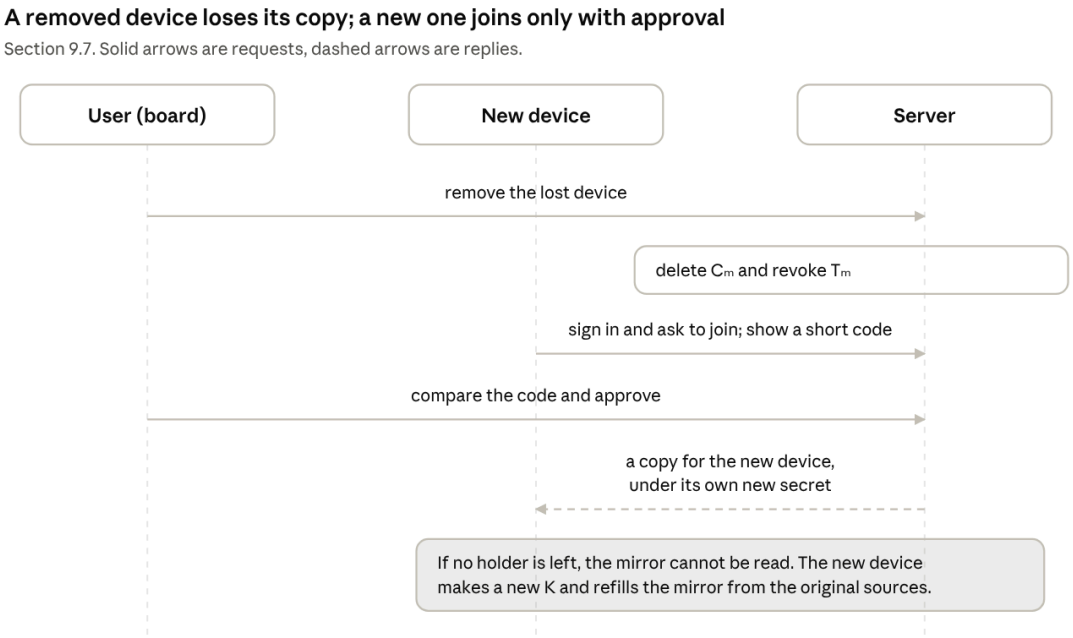


Figure 11.7: a lost device, and a new one · 3 parties

Revoking a device is deleting one copy; a new device needs approval from a holder the user still has.

11.8 9.8 Disconnect, sign out, delete

- Disconnect an AI: delete C_a, and revoke its access and refresh tokens.
- Sign out on the device: revoke T_m and the board sessions it opened, and delete their copies.
- Delete the account: delete every copy and every row. The Vault deletes K from the Keychain. Backups still hold locked copies until they expire, within 30 days, so a holder secret that survives the deletion could open them until then.

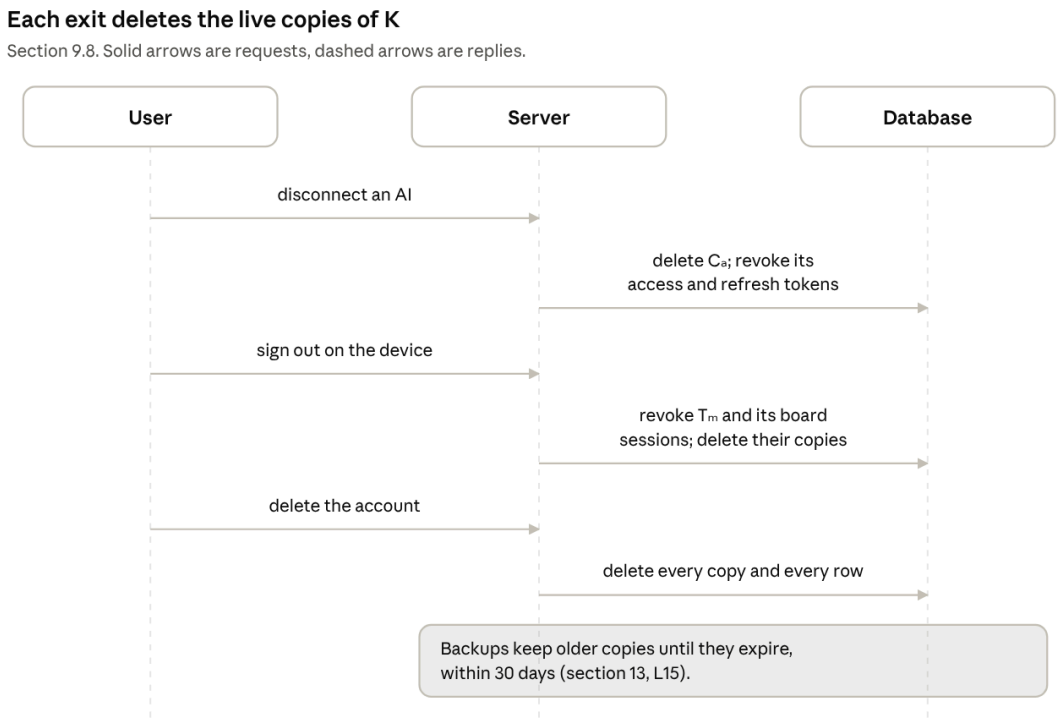


Figure 11.8: disconnect, sign out, delete · 3 parties

Every exit works by deleting the live copies of K. Backups keep older copies until they expire, within 30 days.

12. 10. Searching encrypted data

Search runs over keyed 64-bit trigram tags, so the database index never holds plaintext. The price is a leakage profile, listed below, and it is the design's main weakness against a storage thief (A1).

Construction

1. k_s = HKDF-SHA256(K, info = "search", version). A separate key, k_i, is derived the same way for identity tags.
2. For each indexed field (message text, sender, sender name, chat name, participants, alias names), every distinct three-character fragment t gives the tag $\text{trunc}_{64}(\text{HMAC-SHA256}(k_s, \text{field} \| t))$.
3. A row's tags are stored as a `bigint[]` column with a GIN index. They replace today's pg_trgm indexes.
4. A query's regex is reduced to the fragments any match must contain, the approach of Russ Cox's trigram index. Those fragments become tags, and the GIN index returns candidate rows.
5. The server decrypts the candidates and runs the real regex. A truncation collision only adds a false candidate, which this check removes.
6. A pattern with no required fragment, such as `.*`, falls back to a bounded scan of the most recent rows.
7. Handles, addresses and names are normalized and tagged under k_i, so conversations and the people graph can be grouped without plaintext.

What an adversary with the database learns

Leak	Who sees it	Severity
Each row's number of distinct fragments, roughly its length	A1, A2	Low
Which rows of one account share a fragment	A1, A2	Medium. Frequency analysis and known plaintext can recover common fragments: see the leakage-abuse and inference attacks in section 18.
Which tags a live query touched (the access pattern)	Anyone watching the database while queries run	Medium
Timestamps, row counts, sizes	A1, A2	Low to medium
Anything across accounts	Nobody	None: the tag keys differ per account

A concrete case: a thief who also knows a newsletter that an account received can mark that newsletter's tags and look for them in the account's other rows. Section 16, question 5, asks whether padding or bucketing the tag sets is worth its cost.

13. 11. Performance

The design adds an estimated few milliseconds to each authorized request, mostly the regex recheck. K is opened in memory once per request with no network call, and only the rows in the answer are decrypted.

Step	Work	Estimated cost	Compared with today
Find the holder	One indexed lookup by token hash	Under 1 ms	The same lookup as today
Unwrap K	HKDF and one key unwrap, in process	Microseconds	New, negligible
Search	GIN lookup on keyed tags	Milliseconds	The same index type as today's trigram search
Decrypt the answer	AES-256-GCM with hardware acceleration	About 1 µs per KB	New, negligible for 50 rows
Recheck	The real regex over the candidates	About 1 ms	pg_trgm already rechecks inside Postgres today
Debug access only	One Cloud KMS decrypt	A network round trip	New; support requests only

These are estimates, not measurements. Before anything switches over, the search work will measure them on a synthetic account the size of the largest real one, about 1.6 million messages, and publish the results here.

14. 12. Security analysis

The threats below are sorted by STRIDE category and tied to the trust boundaries drawn in section 3. The largest residual risk is T3: until level 3, a malicious server build can read what authorized requests decrypt.

Trust boundaries: TB1 device to server. TB2 AI provider to server. TB3 browser to server. TB4 server to storage (Cloud SQL, Cloud Storage, backups). TB5 server to Cloud KMS. TB6 support staff to server. TB7 external sources to server. TB8 build pipeline to the running server.

#	STRIDE	Boundary	Threat	Mitigation	Residual risk
S1	Spoofing	TB2, TB3	A stolen holder token is used to act as that holder	Per-holder copies; revocation deletes one row; the board token lives only in an HttpOnly cookie; token expiry (question 13)	Access until revoked or expired
S2	Spoofing	TB2	A rogue OAuth client is approved	PKCE S256, an exact redirect allowlist, the consent page, and an email on every new connection (planned)	A user tricked into approving
S3	Spoofing	TB1	A rogue device joins the account	The user compares a code on an existing holder before approving (question 4)	Phishing of the approval step
T1	Tampering	TB4	Ciphertext is moved between rows, columns or accounts	AES-GCM associated data binds account, table, column and row	An older value of the same row can be replayed
T2	Tampering	TB4	Tags are altered to hide or surface rows	Every candidate is decrypted and rechecked	Hiding rows from search
T3	Tampering	TB8	A malicious build is deployed	Public key-handling code, reproducible signed builds, a pub-	Until level 3, that build sees plaintext during requests

#	STRIDE	Boundary	Threat	Mitigation	Residual risk
				lic log; attestation at level 3	
R1	Repudiation	TB6	Support denies having read an account	Every KMS decrypt is in Cloud Audit Logs and shown to the user	Expiry is enforced by the server, not KMS
I1	Information disclosure	TB4	Database, backup or bucket theft	Content encrypted; tags keyed per account; tokens stored as hashes	Metadata and tag overlap (section 10); backups taken before cutover
I2	Information disclosure	TB4	Content written to logs	A key context that never logs; a test that scans the server's log calls for content	A logging change that the test misses
I3	Information disclosure	In process	Memory read on a compromised server	K lives for one request	Not defended until level 3
I4	Information disclosure	TB7	Source tokens used to reread the originals	Google and Slack tokens wrapped under K	None, once built
D1	Denial of service	All	Every holder is lost	The mirror refills from the original sources	Data that exists only on the server, if stored under K
D2	Denial of service	TB5	Cloud KMS is unavailable	KMS is used only for Debug access	Support is delayed
E1	Elevation of privilege	TB2, TB3	A bug serves another account's rows	Per-account keys: the rows do not decrypt under the caller's K	A bug in the key context itself
E2	Elevation of privilege	TB6	Staff use admin routes to read accounts	Admin routes need a key like every other route, and return ciphertext without one	Debug access, while the user has it on

15. 13. Residual risks and known limitations

The design leaves fourteen known weaknesses, listed here so no reader has to discover them. The first three matter most.

#	Limitation	Consequence	What would close it
L1	The server holds K and plaintext in memory during each authorized request	A compromised server or a malicious build can copy them (T3, I3)	Attestation, level 3
L2	Search tags leak which rows of an account share a fragment	A storage thief with some known plaintext can infer more (section 10)	Padding or bucketing tag sets (question 5)
L3	Backups taken before cutover hold plaintext	Readable by a backup thief until they expire, within 30 days	Time
L4	AI providers see what they ask for	Their handling is outside this design	None here
L5	Metadata stays visible: timestamps, sizes, counts, access times	Traffic analysis of one account	Not planned
L6	The promise depends on locking the source tokens	Without it, the operator could re-read Gmail, Drive, Calendar and Slack directly	Locking Google and Slack tokens under K, before launch
L7	Whoever controls the device at first run controls K	Malware at enrollment owns the key	A hardware-backed or user-presence gate on the OS keystore item (planned, not designed here)
L8	A board session opened from an unlocked device has full device rights	Someone at the device can open the mirror	Narrower session scopes
L9	A hijacked browser session can approve a new AI	The attacker becomes a key holder	An email to the user on every new connection (planned)
L10	Refresh tokens carry the AI's secret	A leaked refresh token is a long-lived key	Refresh-token expiry and binding (questions 7 and 13)
L11	Debug access expiry is enforced by the server, not by Cloud KMS	A server bug could extend support access	KMS-side conditions (question 8)

#	Limitation	Consequence	What would close it
L12	Arrivals are in server memory, in the clear, before sealing	A compromised server sees new mail as it arrives	Attestation, level 3
L13	Background features that need content (triage, filing, briefs) cannot run without a caller	Board accounts are not covered by the first phase	Question 12
L14	No recovery key exists	Losing every holder loses data that exists only on the server	A user-held recovery key, if wanted
L15	Revocation and account deletion are enforced by the server, not by cryptography	A backup taken earlier still holds the locked copies, so a holder secret that survives can open it until the backup expires, within 30 days	Rotating K on revocation (question 3), or shorter backup retention

This is server-side encryption, not end-to-end encryption. Readers who need the server never to see plaintext should treat L1 as decisive until level 3 ships.

16. 14. Verification and transparency

Encryption alone still asks users to trust the operator. Three levels make the claims checkable from outside; levels 1 and 2 are proposed for the first release, and level 3 as its own project.

Level	The operator does	Outsiders can check	Does not prove
1. Review	Publishes the key-handling code and this design. Commissions an independent cryptography review and publishes the report.	That the design and the code are sound	That the server runs that code
2. Transparency	Builds each release reproducibly (SLSA), signs it with Sigstore cosign, records it in the Rekor public log, and reports the running image digest on `/health`. A Verify page in the client lists every key holder and every support use, from the KMS audit log.	That each release is a logged build, and who can open their key	That the running process is that build
3. Attestation	Runs the server in Google Confidential Space on AMD SEV-SNP. Each locked copy is wrapped a second time under a server key that Cloud KMS releases only to the attested image digest. The client checks the attestation before it issues a token.	That the server runs that build and cannot be logged into, by the operator included. A stolen token is useless on any other build.	Moving the server from an SSH-managed VM to a sealed container is a project of its own

The chain from source to running server

1. Public source: the key-handling code, the token handling, the encrypted column type and the search tags.
2. A reproducible build: the same source always yields the same image digest.
3. Signed and logged: each release digest is signed and entered in a public transparency log.
4. An attested server: a confidential VM proves its digest, and keys are released only to it.

At each link, someone outside can check without trusting the operator. A reviewer reads the source. Anyone rebuilds it and compares the digest with the public log and with `/health`. The user's client verifies the attestation.

17. 15. Implementation status and roadmap

None of the twenty pieces below is built. They group into roughly ten pull requests, and client-only accounts switch first.

#	Group	Piece	What it does	Technology
1	Keys and tokens	Account key	Made by the client on first run and kept in the OS keystore; one per account	Native client code, platform cryptography library, OS keystore
2	Keys and tokens	Locked copies	One copy per holder, in a table listing its holder, creation time and expiry	HKDF-SHA256, AES key wrap (RFC 5649)
3	Keys and tokens	Tokens that carry the secret	Device tokens, OAuth tokens and the board's session cookie each carry their holder's secret; the server keeps only a hash	OAuth 2.1, SHA-256
4	Keys and tokens	Request key context	Opens K once per request and holds it only for that request; a route that needs K and lacks it fails, never falling back to plaintext	FastAPI dependency, contextvars
5	Storage	Encrypted column type	One column type for text and JSON, on every content column, bound to account, table, column and row	SQLAlchemy TypeDecorator, AES-256-GCM
6	Storage	Content coverage test	Reads the models and fails if any content column is stored in the clear, so a column added later cannot slip through	Static (AST) test
7	Storage	Encrypted files	Attachments, contact photos and uploads encrypted before	Cloud Storage, streaming AES-256-GCM

#	Group	Piece	What it does	Technology
			upload, in chunks for large files	
8	Search	Keyed search tags	Replace today's trigram indexes (section 10)	HMAC-SHA256, Postgres GIN
9	Search	Query planner	Regex to required fragments, candidate lookup, decrypt and recheck, bounded fallback scan	Python regex parser
10	Search	Keyed identity tags	Handles, addresses and names matched by keyed tag, so grouping and the people graph keep working	HMAC-SHA256
11	No request open	Sealed arrivals	Slack sync, inbound email and the Telegram bot seal each arrival to the account's public key	HPKE (RFC 9180), X25519
12	No request open	Key-gated jobs	Work that needs content runs only inside a request that holds K	The existing job runner
13	No request open	Locked source tokens	Google and Slack tokens wrapped under K	AES key wrap
14	Access and cutover	Debug access	The Settings control, the support key, an expiry, revocation, and a list of every use shown to the user	Cloud KMS HSM, RSA-OAEP, Cloud Audit Logs
15	Access and cutover	Operator routes	Admin export and raw SQL routes need a key like everything else, and return ciphertext without one	Existing routes
16	Access and cutover	Backfill and cutover	At each device's next check-in: encrypt existing rows, build tags, then drop the plaintext columns and old indexes	Alembic, a batch job

#	Group	Piece	What it does	Technology
17	Access and cutover	Public documents	The privacy policy, terms, and the key-custody statements	Documents
18	Portability	Written wire format	Versioned envelopes for locked copies, content, sealed arrivals and tag derivation, with test vectors every client must pass	A specification
19	Portability	One shared core library	The cryptography written once, so a client on another platform writes only its key storage	Rust, with Swift, C# and Linux bindings
20	Portability	Device enrollment	A second device, on any platform, joins through an existing holder (question 4)	—

Why client-only accounts go first. The server already does no background AI work for them, so it reads their content only inside a request or an upload. Accounts that use the board's background features follow once question 12 is settled.

Other platforms. The server never knows what kind of device a holder is, so a client on another platform producing the same bytes is one more holder. Only key storage differs: Credential Manager or DPAPI with a TPM on Windows, and the Secret Service with TPM2 sealing on Linux. Linux desktop keyrings are often readable by any process in the session, so a Linux client needs TPM sealing or a passphrase to match the guarantee of the other platforms.

Fixes to today's code. Writing this document surfaced weaknesses in today's token handling that would undercut the design. They are being fixed before any piece above is built. Under coordinated disclosure, each is listed in the revision history once its fix ships.

18. 16. Questions for reviewers

These fourteen points are where review is wanted most. Comment on a question directly, or anywhere else in the document.

1. **Token format.** A holder presents T_h and s_h together in one bearer value. Is that sound, or should s_h travel separately so that a leaked lookup token alone is useless?
2. **Nonce budget.** K encrypts every content value for the life of the account, with random 96-bit GCM nonces. NIST SP 800-38D limits random nonces to 2^{32} uses per key, and the largest account holds about 1.6 million messages across several columns. Per-table keys derived by HKDF, AES-GCM-SIV, or XAES-256-GCM?
3. **Rotating K.** Re-encrypt the mirror, or keep content under data keys wrapped by K, so that rotation re-wraps only keys?
4. **A second device.** Is comparing a short code on an existing holder enough to admit a new device? Should the new device receive K itself, and over what channel?
5. **Search leakage.** Is a keyed 64-bit trigram index acceptable for this threat model? Are padded tag sets or bucketed fragments worth their cost in index size and false candidates?
6. **The board session's copy.** Make it when the connect code is minted, deleting it if unredeemed, or when the code is redeemed, which needs the code to carry a secret?
7. **Refresh.** Carry s_a inside the refresh token, or re-wrap C_a under a fresh secret at each refresh?
8. **Debug access expiry.** Is deletion by the server enough, or should the KMS decrypt permission carry an IAM condition that ends at the same time?
9. **HPKE on the server.** Which maintained HPKE implementation should a Python 3.11 server use?
10. **Verification for the first release.** Are levels 1 and 2 enough to ship, with level 3 to follow?
11. **Wire format.** Beyond versioned envelopes and test vectors, what must the specification fix so that independent clients interoperate safely?
12. **Background features.** Triage, filing and briefs need content when no holder is calling. Run them only on a holder's call, on the user's device, or not at all for encrypted accounts?
13. **Token lifetimes.** Now that every token carries a key, what expiry and renewal should device, access, refresh and session tokens have?
14. **Recovery.** Should users be offered a recovery key they hold themselves, accepting that it can be lost or stolen?

19. 17. Vulnerability disclosure

Report a security problem in the running service privately, never in a comment here. Comments on this document are for the design.

- **Where to report:** open question; a reporting address is still to be named.
- **What the operator's security policy commits to:** an acknowledgement within 2 business days, an assessment within 5, and a fix for a critical or high-severity issue within 7 days of confirming it, or a stated reason and date.
- **Safe harbour:** good-faith research that follows the policy is treated as authorized, with no legal action.
- **Please include:** what the issue is and what an attacker gains, exact steps to reproduce, the host you tested, and anything you reached that was not yours.
- **Out of scope:** denial-of-service and high-rate automated scanning, social engineering, physical attacks, and third-party services the operator does not run.
- **Payment and credit:** there is no bug bounty. You are credited by name in the fix if you want to be.

Open question: a `security.txt` (RFC 9116) on the service's domain, so researchers find this contact without the document.

20. 18. References and glossary

Standards

- RFC 2104, HMAC
- RFC 3552, Guidelines for Writing RFC Text on Security Considerations
- RFC 5649, AES Key Wrap with Padding
- RFC 5869, HKDF
- RFC 7636, PKCE
- RFC 7748, Elliptic Curves for Security (X25519)
- RFC 8017, PKCS #1 v2.2 (RSA-OAEP)
- RFC 9116, security.txt
- RFC 9180, Hybrid Public Key Encryption
- RFC 9700, Best Current Practice for OAuth 2.0 Security
- The OAuth 2.1 Authorization Framework (Internet-Draft)
- NIST SP 800-38D, Galois/Counter Mode
- NIST SP 800-57 Part 1 Rev. 5, Recommendation for Key Management
- Fernet specification
- SLSA, Sigstore, Google Confidential Space

Research on searchable encryption leakage

- Cash, Grubbs, Perry and Ristenpart, “Leakage-Abuse Attacks Against Searchable Encryption”, ACM CCS 2015.
- Naveed, Kamara and Wright, “Inference Attacks on Property-Preserving Encrypted Databases”, ACM CCS 2015.
- Russ Cox, “Regular Expression Matching with a Trigram Index”, 2012.

Method: threat categories follow STRIDE, as in Adam Shostack, *Threat Modeling: Designing for Security* (Wiley, 2014).

Glossary

Term	Meaning
Mirror	The server-side copy of an account’s mail, messages, calendars, contacts and files
Client	The desktop app that reads the user’s sources on the device and uploads the mirror
Board	The web interface that shows the mirror in a browser
Holder	Anything that can open K: the device, a connected AI, a board session, a personal access token, or support during Debug access
K, account key	The 256-bit key that encrypts one account’s mirror
Locked copy, C_h	K wrapped under a key derived from one holder’s unlock secret
Unlock secret, s_h	A 256-bit random secret carried only inside a holder’s token
Keyed tag	A truncated HMAC of a text fragment or an identity, under a key derived from K

Term	Meaning
Sealed arrival	A message encrypted to the account's public key while no holder is calling
Debug access	A user-controlled, time-limited grant that lets support open the account through Cloud KMS
MCP	Model Context Protocol, the interface AI assistants use to query the mirror

Appendices

A. Appendix A. Submission process

A change counts as submitted, and is credited, only when it arrives signed by its author through one of the channels below. A name typed into a commit, an email or a comment is never evidence of who wrote it.

The rule. Attribution comes from a cryptographic signature, or from an account the platform has authenticated, never from text. Git author names, `Co-authored-by` lines, email From headers and names in a document can all be typed by anyone.

Channels

Channel	Counts as	Attributed to	Verified by
GitHub pull request, every commit signed	An official text change	The GitHub account whose registered GPG or SSH key signed each commit	GitHub's Verified mark, plus a required check that fails on any commit without a verified signature
GitHub issue, from the submission template	An official comment or objection	The GitHub account that opened it	GitHub's sign-in. At intake the issue's text is hashed into the submission log, so a later edit shows.
Signed email to the submissions address	An official submission, for people without GitHub	The holder of the signing key or identity	An S/MIME or OpenPGP signature, or a Sigstore signature bound to a verified email. The editor files it as an issue with the signature attached, so anyone can check it.
Google Doc or Claude Doc comment, unsigned email, DOCX	Discussion only	Not attributed	It becomes a submission when its author files it through a channel above

Steps

1. Discuss anywhere. Discussion is welcome, but it is not a submission.
2. Submit through a signed channel. A pull request carries a `Signed-off-by` line under the Developer Certificate of Origin, which licenses the change under CC BY 4.0. That line is a licence, not proof of identity.
3. Intake. Each submission gets a number, from S-0001, in `SUBMISSIONS.md`: its channel, its account or key fingerprint, its time, and the SHA-256 of its content.
4. Disposition. The editor records accepted, declined with a reason, or deferred, as a signed commit naming the submission number.
5. Merge. Pull requests merge with a merge commit only. Squash and rebase merges rewrite the submitter's commits, so their signatures would no longer cover what lands. The editor's own changes are separate signed commits, so readers can always tell the submitter's words from the editor's.
6. Release. Each version is a signed Git tag. Its commit hash is recorded in Sigstore's public Rekor log and the tagged tree is saved to Software Heritage, so history cannot be quietly rewritten later, by the editor or by GitHub.
7. Credit. The contributor list is generated from the verified signatures in the history, never typed by hand.

Attacks on attribution, and what stops them

Attack	Control
Committing under someone else's name or email	Unsigned commits are refused. Submitters should turn on GitHub's vigilant mode, so anything unsigned under their email shows as Unverified.
Claiming in a description to be someone else	Attribution comes from the signing account, never from text
The editor changing a submitter's text before merging	Submitter commits merge unchanged; the editor's edits are separate signed commits, visible in the diff
Forged `Co-authored-by` or `Signed-off-by` lines	Never used for attribution
Spoofed email	Only signed email counts
Rewriting history	A protected main branch: signed commits required, no force-push, no deletion. Tag hashes in Rekor. Copies in Software Heritage.
Deleting or editing a comment in a shared document	Those comments are discussion, not submissions
Many accounts posing as broad support	Submissions are weighed on their merit, never counted as votes

What it cannot stop. If a submitter's own account or signing key is stolen, the thief can submit as them. Use two-factor authentication and, ideally, a hardware security key. A submitter who finds a submission they did not make can disown it with a signed statement, and it is marked disputed.

Open question: the public repository, its branch protection, the signature check, the issue template and the submissions address must exist before the comment period opens.